ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ
TECHNICAL UNIVERSITY OF CRETE

Electrical and Computer Engineering

Number Theory & Cryptography
Assignment 1

Course Instructor

Prof. Georgios Karystinos

Assignment Supervisor

Ioannis-Leonidas Steiakakis
isteiakakis@tuc.gr

Assignment Instructions

For the programming parts of the assignment, the accepted programming language is Python 3 (as `.py` files, not Jupyter notebooks).

For tasks requiring code implementation, name the script file for each task appropriatelly, as shown in the following tree diagram depicting the file structure in your deliverable.

For a task requiring code implementation, you may copy-paste a script from a previous task and use it as a starting point.

Try to follow good programming practices.

- You may refactor code into functions for modularity and logical abstraction for a cleaner code in the main script file of a task.
- Please use appropriate comments: clear and explanatory, but not overly verbose or unnecessary (e.g., avoid commenting on self-explanatory function calls and easily understandable lines of code).

In code simulation tasks, the indicative values given for parameters are just indicative. Do not hardcode them. Your code should be parameterized by those parameters and should work for other (appropriate) values, too.

You may use an immediate result (i.e., without proof) directly from the course notes, provided you appropriately reference[1] it, of course. Make sure to adjust the notation to match that of the assignment prompt and your deliverable report.

Important! Your solutions must strictly follow the theory from course notes. Any non-trivial concepts, whether in theory or code, must be thoroughly explained. For theoretical aspects, provide clear reasoning and justification. For code, ensure that your implementation is well-commented and reflects the underlying theory. Theoretical ideas applied to your code should also be clearly justified and well-documented. Lack of proper explanation may lead to point deductions.

Also important! Your deliverable must follow the structure shown below. `etc/` is a directory for any other file you may want to include. `util.py` is a file with functions implemented by you which can be used in the task scripts by importing it (`import util`). `task*.py` files are your implementations of each task; of course an implementation can be moved in a function in `util.py` and just demonstrated in the task script. They must run as `` `python3 task`⟨*id*⟩⟨*suffix*⟩`.py` `` (no extra command line arguments), where *id* is the task ID and *suffix* is optional for tasks with multiple script files.

[1] That is, clearly indicate the source of the result.

```
report.pdf
src/
|-- etc/
|   |-- intercepted_data.json     (used in tasks 5e-5i)
|   |-- ...
|-- util.py
|-- task2a.py
|-- task3a.py
|-- task3b.py
|-- task4b.py
|-- task4c.py    (if needed)
|-- task5d_client.py
|-- task5d_server.py
|-- task5e_client.py
|-- task5e_server.py
|-- task5e_hacker.py
|-- task5f_client.py
|-- task5f_server.py
|-- task5f_hacker.py
|-- task5g_client.py
|-- task5g_server.py
|-- task5g_hacker.py
|-- task5h_client.py
|-- task5h_server.py
|-- task5h_hacker.py
|-- task5i_client.py
|-- task5i_server.py
|-- task5i_hacker.py
|-- task5j_client.py
|-- task5j_server.py
```

Contents

1 Preliminaries **1**
Task 1.a (0%) 1

2 Euclidean Algorithm **3**
Task 2.a (5%) 3

3 Hill Cipher **4**
Task 3.a (0%) 4
Task 3.b (10%) 4

4 A Decentralized Authentication Protocol **5**
Task 4.a (5%) 5
Task 4.b (10%) 5
Task 4.c (15%) 6

5 A User Authentication Protocol **7**
Task 5.a (4%) 9
Task 5.b (1%) 9
Task 5.c (5%) 9
Task 5.d (15%) 9
Task 5.e (0%) 10
Task 5.f (5%) 10
Task 5.g (10%) 10
Task 5.h (2%) 10
Task 5.i (10%) 10
Task 5.j (3%) 11

1 Preliminaries

Let's start with an attempt to comprehend the size of large numbers.

Task 1.a (0%)

- What is the life duration of a human (assume 100 years) in nanoseconds expressed as a power of 2?
- What is 10 times that duration (i.e., 1000 years) in nanoseconds expressed as a power of 2?
- What is the age of the universe (13.8 billion years) in nanoseconds expressed as a power of 2?

No need for extra commentary, just write down the final results.

Now, we should have some intuition about how likely an event with probability 2^{-x} (for $x > 0$) is to occur.

Definitions and Instrustions for the Assignment

- $\mathcal{T}_n \triangleq \{a \in \{1, \ldots, n\} \mid (a, n) = 1\}$ for any $n \geq 1$.
- $s \xleftarrow{\$} S$ denotes sampling an element s uniformly at random from the set S.
- To sample from a proper subset of $\mathbb{Z}_n$, first sample from $\mathbb{Z}_n$ and then apply rejection sampling.

Rules to Follow

For algebraic operations, you must use only Python operators, such as:

- Arithmetic operators: `+`, `-`, `*`, `/`, `//`, `%`, `**`
- Bitwise operators: `&`, `|`, `^`, `~`, `<<`, `>>`
- And, of course, their assignment variants, e.g., `+=`, `-=`, ...

Also, the following built-in functions are allowed: `abs`, `sum`, `min`, `max`

You are not allowed to import anything other than your modules and the following:

```
import json
import socket
from hashlib import sha512
from secrets import randbelow, randbits
from Crypto.Util.number import getPrime # dependencies: `pip3
    install pycryptodome`
```

Modules like `os`, `shutil`, `sys`, etc, are also acceptable.

Lists and dictionaries are allowed.

The restriction on operators and modules is intended to avoid using ready-made implementations in cases where the functionality must be implemented by you. If you wish to use something that you believe does not violate this, but is not listed above, you may ask for clarification.

2 Euclidean Algorithm

Task 2.a
(5%)

Implement in code the *accelerated* (standard and extended) Euclidean algorithm. Explain whether it aligns with the theory presented in the notes; if not, describe any modifications to the theory that may be necessary.

Implementation detail: Your function must accept a third optional parameter named `extended` which defaults to `False`. If it is `False`, the standard version of the Euclidean algorithm is executed; otherwise, the extended version is executed.

3 Hill Cipher

Task 3.a (0%)

Implement in code the Hill Cipher ignoring the constraint on the keys. Does it work correctly?

It is important in cryptography that keys are generated as randomly as possible so that they are difficult for attackers to predict.

Your implementation must consist of the `keygen`, `encrypt`, `decrypt` functions, and a demonstration.

Task 3.b (10%)

Use your implementation from Task 3.a fixing the constraint.

M is the modulus and a, b, c, d constitute the key.

The constraint $(ad - bc, M) = 1$ does not allow for simple sampling in key generation. However, we have to keep them as random as possible.

So, we will generate them this way:

1. Sample some $k \xleftarrow{\$} \mathcal{T}_M$ (since $(ad - bc, M) = 1 \overset{\text{Lemma 1.28}}{\iff} (r_M[ad - bc], M) = 1 \iff r_M[ad - bc] \in \mathcal{T}_M$)
2. Choose randomly whether $[a, d]$ or $[b, c]$ will be used in the next step.
3. If the choice was $[a, d]$, then $a \xleftarrow{\$} \mathbb{Z}_M, d \xleftarrow{\$} \mathbb{Z}_M$.
   - Think how to generate the rest parameters using the aforementioned sampled values. Do not forget that being as random as possible is important. Avoid determinism as much as possible and sample values as randomly as possible.
4. If the choice was $[b, c]$, then proceed accordingly.

4 A Decentralized Authentication Protocol

A cryptographic protocol designer wishes to design a protocol where a party A may join a network only if A possesses a secret value x.

The designer considered dangerous a centralized approach where a single authentication server knows x (and a candidate party sends through a secure channel a value to the server to be compared with x), because the server may be untrustworthy and could share x.

So, the designer decentralizes the authentication process by having many authentication servers where each server knows only a small part of x.

Assume ℓ authentication servers. Let $n = \prod_{i=1}^{\ell} p_i^{k_i}$ (the product is its canonical form) and $x \in \mathbb{Z}_n$.

For $i \in \{1, \ldots, \ell\}$, server i knows only $r_{n_i}[n_{i-1}^n x]$ and keeps it secret if it is trustworthy, where $n_i = p_i^{k_i}$ and $n_0 = n_\ell$.

When a candidate party (which possesses some $x' \in \mathbb{Z}_n$) wishes to authenticate, the party sends through a secure channel to the authentication server i the value $r_{n_i}[n_{i-1}^n x']$, for every $i \in \{1, \ldots, \ell\}$. If every server finds the received value equal to its own, then the party is accepted; otherwise, it is rejected.

Everything about the protocol design and every parameter (except for x) are publically known.

Task 4.a (5%)

Describe the possible values of x that each authentication server i can infer. Is the set of these values a complete set of residues modulo n_i?

Task 4.b (10%)

Implement a code simulation of the described protocol.

Implementation instructions:

- Exponentiations inside the modulo operation must be computed efficiently. Multiplication as well.
  - "Efficiently" means avoiding multiplications between numbers of large absolute value.
- Indicatively, $\ell = 20$.
- Sample the primes using `getPrime`, indicatively, use 256 bits for each.

- Sample k_i for each i uniformly in $\{1, \ldots, k_{\max}\}$, indicatively, $k_{\max} = 10$.

Task 4.c
(15%)

Is the protocol secure if every server is untrustworthy? If yes, prove it theoretically; if no, design theoretically and implement in code (using your implementation from Task 4.b) an attacker that extracts x.

Consider the bruteforce attack as infeasible.

5 A User Authentication Protocol

In this exercise, we will implement a user authentication protocol.

Protocol parameters (stored in server):

- $n \in \mathbb{Z}_{>0}$ is the modulus
- $t \in \mathbb{Z}_{>0}$ is the number of times that the login challenge will be repeated

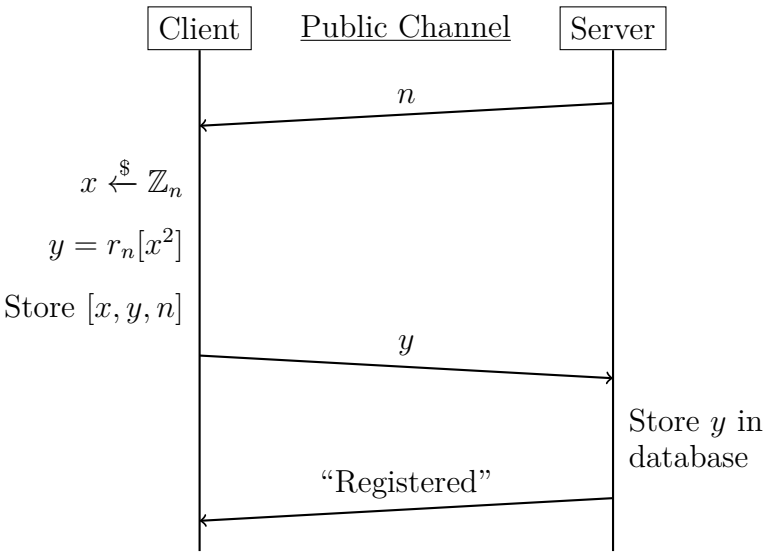

Figure 1: Registration

The server needs to check whether

$$x^2 \equiv y \pmod{n} \tag{5.1}$$

holds for a login attempt to be successful.

Assume that, for known y, n, and unknown x, (5.1) is hard to solve for x.

x is the secret of the user. After registration, when a user wishes to log in, they need to verify themselves, i.e., to prove that they possess x, y, n such that (5.1) holds and y is registered in server's database. Sending x as is to the server over the public channel would enable the verification process, however, it would also expose x.

Imagine y as the username and x as the password.

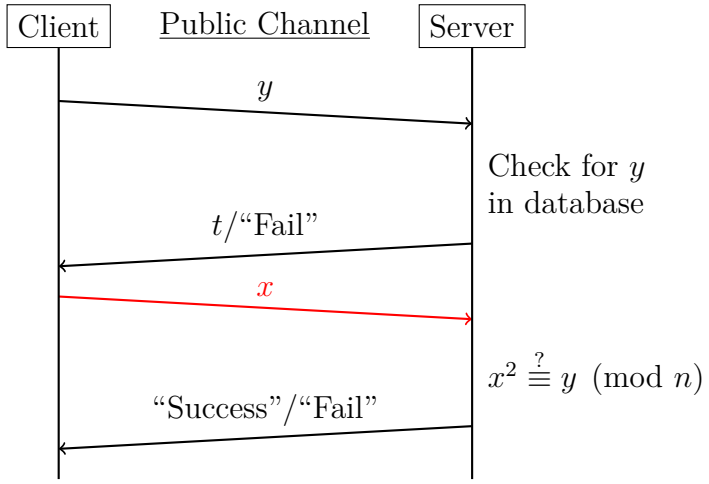

Figure 2: Dangerous login (not to be used in any task, just showing it)

To address this problem, the following protocol has been designed.

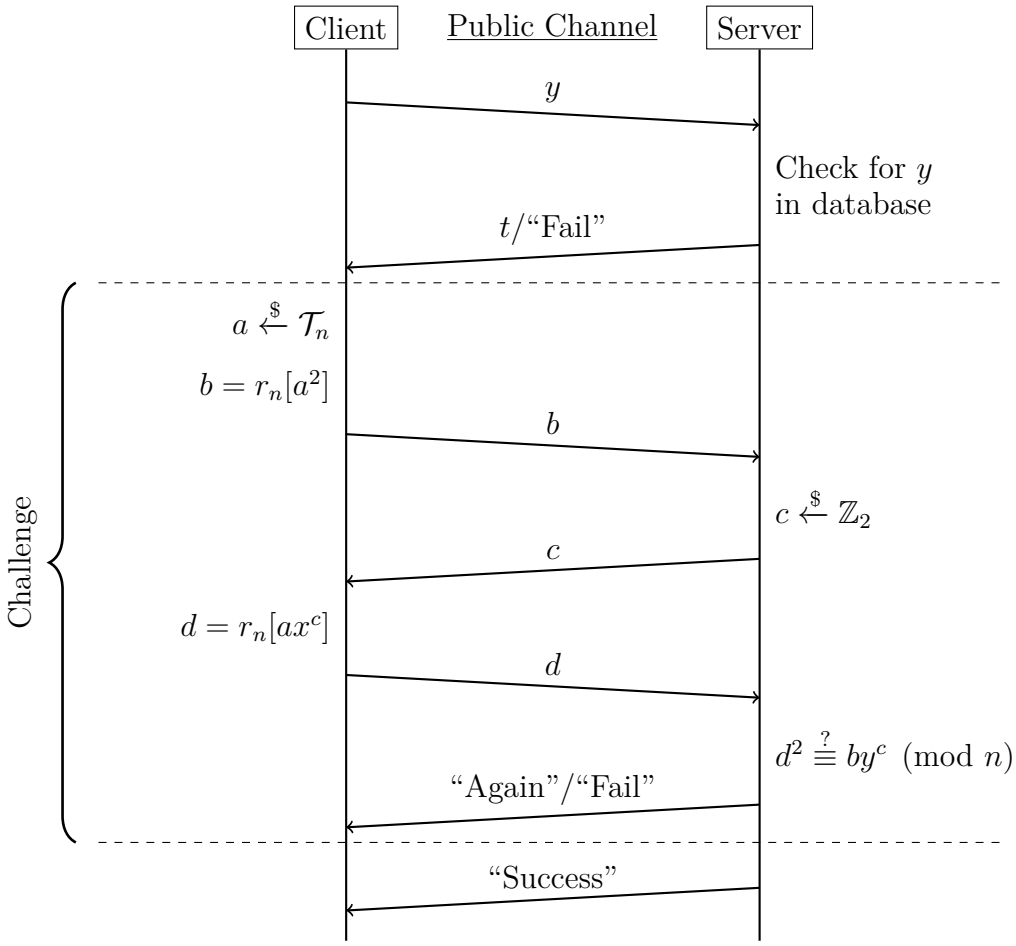

Figure 3: Login

Define $0^0 = 1$.

Implementation details:

- Client and server are implemented in different files and communicate using TCP.
- Server runs all the time, listening at 127.0.0.1:8080.
- A single client may be served at a time, no need for multithreading.
- When a client connects to the server, the server sends an initial prompt asking the client to register, log in, or exit.
  - If the client wishes to register or log in, then the server follows the corresponding process. When this process finishes (successfully or not), the server reverts to the initial prompt state.
  - If the client wishes to exit or if it drops the connection, then the server is again available for any possible incoming connections.
- For the login process:
  - If at any point the login verification fails, then the server sends a "Fail" message and the login process finishes (the server reverts to the initial prompt state).
  - If y is not in the server's database, then the login verification fails. Else, the server sends t and the process continues.
  - For a successful login verification, the challenge is repeated for t successive successful times. If at any time it fails, then the login verification fails.
  - If the login verification is successful, then the server sends a "Success" message and, since our simulation ends here, the server reverts to the initial prompt state.

Task 5.a (4%)

Can an eavesdropper extract x? Justify your answer.

Task 5.b (1%)

Does an honest client pass the login verification? Justify your answer.

Task 5.c (5%)

Can a malicious client pass the login verification? Justify your answer.

Task 5.d (15%)

Implement the protocol in code using $t = 100$ and n a uniformly random 4096-bit integer.

In the following (intermediate) task, we will implement (simulate) the data interception over the public channel.

Assume a hacker has tampered with the client's source code. The malicious code added by the hacker copies any values transmitted over the public channel to a file. The data in the file may be structured, rather than appearing in the raw form of the TCP stream.

Once the client-server communication is finished, the hacker can read the file and interact with the server. The hacker knows how do the register and login protocols work.

Task 5.e
(0%)

Use your implementation from Task 5.d making the appropriate changes in client for sent/received data interception. Also, implement a hacker that (after the communication is completed) reads and parses the intercepted data.

In the following tasks, the attacks must be justified theoretically before implemented.

Task 5.f
(5%)

Use your implementation from Task 5.e changing only $t = 3$ in the server and the hacker. Gain unauthorized access to a registered user's account (i.e., successfully log in as the user).

For the following task, assume that the developer who programmed the client made a mistake in the login process: a is sampled randomly only once, at the beginning of the login process; it is then reused for the following challenges.

Task 5.g
(10%)

Use your implementation from Task 5.e changing only the client as described and the hacker. Gain unauthorized access to a registered user's account.

For the following task, assume that a programming mistake happened in the server login process: c is fixed instead of sampled randomly.

Task 5.h
(2%)

Use your implementation from Task 5.e changing only the server as described (using $c = 0$) and the hacker. Gain unauthorized access to a registered user's account.

Task 5.i
(10%)

Same as Task 5.h, but for $c = 1$.

As a final task, we will make the implementation more realistic by making it username-password based

Task 5.j (3%)

Use your implementation from Task 5.d making the appropriate changes in client and server that enable the client to use username and password.

The changes:

- x is computed using the password that the user will provide.
  - Since $4096/512 = 8$, we will concatenate 8 hash values.
  - For $i = 0, \ldots, 7$, compute $\text{SHA512}(i \| \texttt{password})$ and concatenate them, i.e.,

$$\text{SHA512}(0\|\texttt{password})\Big\|\,\text{SHA512}(1\|\texttt{password})\Big\|\ldots$$

    where $\|$ denotes concatenation of byte sequences, hence the string `password` and the 4-byte integer i must be converted to byte sequences before concatenation. Then, convert this result from a byte sequence to an unsigned integer. This is x.
- In registration, the client enters a username and a password. x is computed from the password as mentioned. Then, alongside y, the username is also sent to the server, and both are stored in server's database as a pair. The client does not store x, y any more.
- In login, the client enters a username and a password. x is computed from the password as mentioned. Then, the client sends y and the username so the server checks for this pair in its database.

`hashlib.sha512` demo:

```
>>> from hashlib import sha512
>>> str_val = "Darling, you got to let me know\n" \
...           "Should I stay, or should I go?"
>>> str_val
'Darling, you got to let me know\nShould I stay, or should I go?'
>>> str_val_bytes = str_val.encode()  # convert to byte sequence
    (`str_val_bytes.decode()` will convert to str)
>>> str_val_bytes
b'Darling, you got to let me know\nShould I stay, or should I go?
    '
>>> hash_val = sha512(str_val_bytes)
>>> hash_val
<sha512 _hashlib.HASH object @ 0x7122609b1670>
>>> hash_val_bytes = hash_val.digest() # convert to byte sequence
>>> hash_val_bytes
```

```
b'\x7fB*mH\xea\x936wN\x85\xa7\x9b\x9e\x92]\xf7\xe8\xf7\xa8\x03\
   x06\xf6\x8eE\xc3j\n\x95\x84\xa6\x9e\x95\xfc\x13\r\xf8cr\xed\
   xe2\x8f9p)\xb2\x9a\xe8\xec:W\x94\xbch\x85\xb0c\xd8?\x1e\x0ey\
   xd0\xaf'
>>> int_val = 65
>>> int_val_bytes = int_val.to_bytes(4, "big")  # convert to byte
    sequence (4 bytes, big endian)
>>> int_val_bytes   # note: 'A' = 65 (ASCII)
b'\x00\x00\x00A'
>>> int.from_bytes(int_val_bytes, "big")  # convert to int (big
   endian decoding)
65
>>> hash_val2_bytes = sha512(int_val_bytes).digest()
>>> hash_val2_bytes
b"\xcc'\xaeb\x04\x8a`\x88\xebF\x98\xd6\x1d0\x97\xce\xfb\xb4&\xfbq
   \xbc\xed\xd2v\xf7\x08\xa6\\@\x94h\xb0h\x95PZ\xc0\xf3\x82\xfd_\
   x99\xa1\x19\xb2\x91\xe4\xf4\x82Z\x01}']\x0f\x18\x02\xbb\xf0K\
   xca\x9d\xa5"
>>> int.from_bytes(hash_val2_bytes, "big")
10692465194933905905625332324539054180322978167057781824991950056
60504892327087242231076949610714720125890063282464990669438906624
4790456556825607550967205
>>> b"abc" + b"def"   # concatenate
b'abcdef'
```